

第五章 移动机器人感知技术

激光传感器定位原理

孙 雷 教授

sunl@nankai.edu.cn

Tel: 13512967601

机器人与信息自动化研究所 (IRAIS)
天津市智能机器人技术重点实验室 (tjKLIR)
南开大学

2026 年 4 月 13 日



主要内容



激光传感器定义及分类

定义

利用激光技术进行测量的传感器。它由激光器、激光检测器和测量电路组成。激光传感器是新型测量仪表，它的优点是能够实现无接触远距离测量，速度快，精度高，量程大，抗光、电干扰能力强等。

利用激光的高方向性、高单色性和高亮度等特点可实现无接触远距离测量。激光传感器常用于长度、距离、振动、速度、方位等物理量的测量，还可用于探伤和大气污染物的监测等。

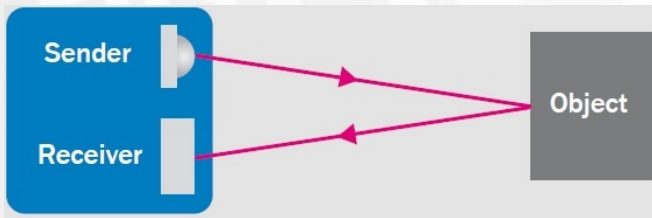
分类

- 1D 激光
- 2D 激光
- 3D 激光

激光测距原理

激光测距一般基于到达时间法 (TOF: Time of Flight):

例如: 为了获得 $3mm$ 的测量精度, 计算机系统需具有什么量级的时间精度?

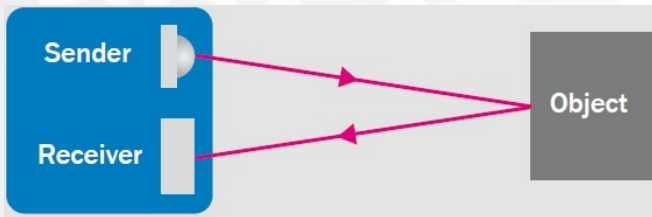


激光测距原理

激光测距一般基于到达时间法 (TOF: Time of Flight):

例如: 为了获得 $3mm$ 的测量精度, 计算机系统需具有什么量级的时间精度?

$$t = \frac{3 \times 10^{-3}}{3 \times 10^8} = 10^{-11}s = 10^{-2}ns$$

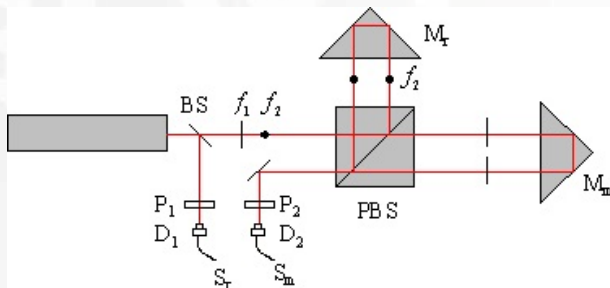


激光测距原理 (2)

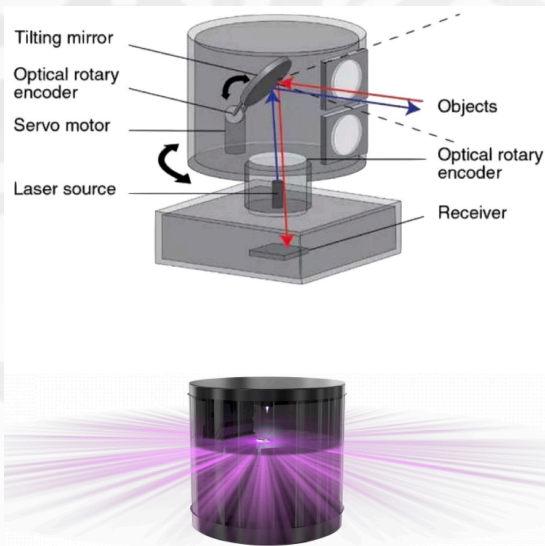
原理图如图所示，左侧为稳频双频激光器，输出两个正交线偏振光，其两频率分量为 f_1 和 f_2 。输出光经分光镜 BS 后，反射光经检偏器 P_1 ，被光电探测器 D_1 接收，形成参考信号；透射光经偏振分光镜 PBS 分为两束，一束被反射到固定角锥棱镜 M_r ，另一束被透射至测量镜 M_m ，两束光反射后经 PBS 合成一束，经检偏器 P_2 ，被光电探测器 D_2 接收，形成测量信号。参考信号和测量信号经前置放大后，送入外差信号处理系统，进行相位测量，从而得出测量镜的位移。

由多普勒原理可知，在双频激光干涉仪中，相位差 φ 与测量镜的位移 ΔL 的关系由下式决定：

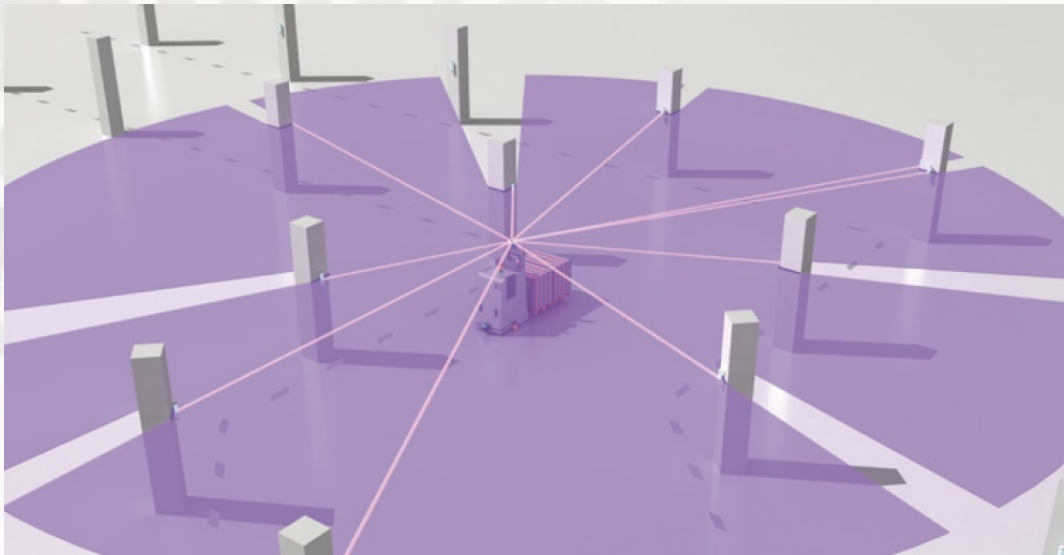
$$\varphi = \frac{360^\circ \times \Delta L}{\lambda/2}$$



2D 激光雷达原理



激光定位原理





What is Least Squares

The method of least squares assumes that the best-fit curve of a given type is the curve that has the minimal sum of the deviations squared (**least square error**) from a given set of data.



What is Least Squares

The method of least squares assumes that the best-fit curve of a given type is the curve that has the minimal sum of the deviations squared (**least square error**) from a given set of data.

Least Squares

Suppose that the data points are $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ where x is the **independent variable** and y is the **dependent variable**. The fitting curve $f(x)$ has the deviation (error) d from each data point, i.e., $d_1 = y_1 - f(x_1), d_2 = y_2 - f(x_2), \dots, d_n = y_n - f(x_n)$. According to the method of least squares, the best fitting curve has the property that:

$$\Pi = d_1^2 + d_2^2 + \dots + d_n^2 = \sum_{i=1}^n d_i^2 = \sum_{i=1}^n [y_i - f(x_i)]^2 = \min. \quad (1)$$



The Least-Squares Line

The least-squares line uses a straight line

$$y = f(x) = a + bx \quad (2)$$

to approximate the given set of data $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, where $n \geq 2$. The best fitting curve has the least square error, i.e.,

$$\Pi = \sum_{i=1}^n [y_i - f(x_i)]^2 = \sum_{i=1}^n [y_i - (a + bx_i)]^2 = \min. \quad (3)$$



Principle of Least Squares

Please note that a and b are unknown coefficients while all x_i and y_i are given. To obtain the least square error, the unknown coefficients a and b must yield zero first derivatives.

$$\begin{cases} \frac{\partial \Pi}{\partial a} = 2 \sum_{i=1}^n [y_i - (a + bx_i)] = 0 \\ \frac{\partial \Pi}{\partial b} = 2 \sum_{i=1}^n x_i [y_i - (a + bx_i)] = 0 \end{cases} \quad (4)$$



Regression Model in Matrix Form

The linear model with several explanatory variables is given by the equation

$$y_i = \beta_1 + \beta_2 x_{2i} + \beta_3 x_{3i} + \cdots + \beta_k x_{ki} + \epsilon_i \quad (i = 1, \cdots, n). \quad (5)$$

For simplicity of notation we write β_1 instead of $\beta_1 x_{1i}$. For purposes of analysis it is convenient to express the model (??) in *matrix form* as

$$Y = X\beta + \epsilon. \quad (6)$$

where

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} x_{11} & x_{21} & \cdots & x_{k1} \\ \vdots & \vdots & & \vdots \\ x_{1n} & x_{2n} & \cdots & x_{kn} \end{pmatrix}, \beta = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_k \end{pmatrix}, \epsilon = \begin{pmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{pmatrix}.$$



Optimization Based on Least Square

From (??), we can obtain that

$$\epsilon = Y - X\beta. \quad (7)$$

the sum of squares of the residuals $S(\beta)$ (a cost function of β) as

$$S(\beta) = \sum \epsilon_i^2 = \epsilon^T \epsilon = (Y - X\beta)^T (Y - X\beta). \quad (8)$$

Thus the goal of the least squares estimator is obtained by $\min_{\beta} [S(\beta)]$.



Optimization Based on Least Square

From (??), we can obtain that

$$\epsilon = Y - X\beta. \quad (7)$$

the sum of squares of the residuals $S(\beta)$ (a cost function of β) as

$$S(\beta) = \sum \epsilon_i^2 = \epsilon^T \epsilon = (Y - X\beta)^T (Y - X\beta). \quad (8)$$

Thus the goal of the least squares estimator is obtained by $\min_{\beta} [S(\beta)]$.

The Optimal Approach Based on Least Square

To attain the goal, we could set derivatives equal to zero, based on (??) that is

$$\frac{\partial S}{\partial \beta} = -2X^T Y + 2X^T X\beta = 0 \quad (9)$$

which gives the normal equations

$$X^T X\beta = X^T Y \Rightarrow \beta = (X^T X)^{-1} X^T Y \quad (10)$$



Triangulation using distance estimates

The location of the unknown node (x, y) can be determined based on measured distance estimates $\hat{d}_i$ to n reference nodes $\{(x_1, y_1), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$. This can be formulated as a least squares minimization problem. Let d_i be the correct Euclidean distance to the n reference nodes, i.e.:

$$\sqrt{(x_i - x)^2 + (y_i - y)^2} = d_i$$



Triangulation using distance estimates

The location of the unknown node (x, y) can be determined based on measured distance estimates $\hat{d}_i$ to n reference nodes $\{(x_1, y_1), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$. This can be formulated as a least squares minimization problem. Let d_i be the correct Euclidean distance to the n reference nodes, i.e.:

$$\begin{aligned}\sqrt{(x_i - x)^2 + (y_i - y)^2} &= d_i \Rightarrow \\ \underbrace{x^2 + y^2}_{\text{nonlinear}} - 2x_i x - 2y_i y &= d_i^2 - (x_i^2 + y_i^2)\end{aligned}$$



Triangulation using distance estimates

The location of the unknown node (x, y) can be determined based on measured distance estimates $\hat{d}_i$ to n reference nodes $\{(x_1, y_1), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$. This can be formulated as a least squares minimization problem. Let d_i be the correct Euclidean distance to the n reference nodes, i.e.:

$$\begin{aligned}\sqrt{(x_i - x)^2 + (y_i - y)^2} &= d_i \Rightarrow \\ \underbrace{x^2 + y^2}_{\text{nonlinear}} - 2x_i x - 2y_i y &= d_i^2 - (x_i^2 + y_i^2)\end{aligned}$$

Remark

How can we utilize the Least-Square in matrix form discussed above to solve this problem ?
(Note: $Y = X\beta + \epsilon$)



Triangulation using distance estimates

In order to formulate the problem as a least squares matrix form, the nonlinear parts in the equation should be eliminated. Thus, we could subtract the i th equation to the n th equation, i.e.:

$$2(x_n - x_i)x + 2(y_n - y_i)y = d_i^2 - d_n^2 + (x_n^2 + y_n^2) - (x_i^2 + y_i^2) \quad (11)$$

that is:

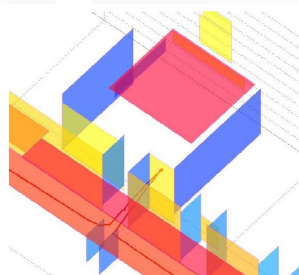
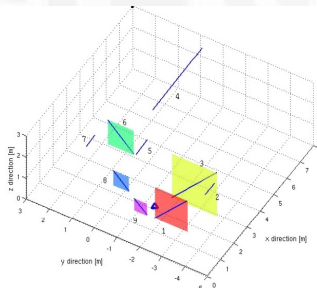
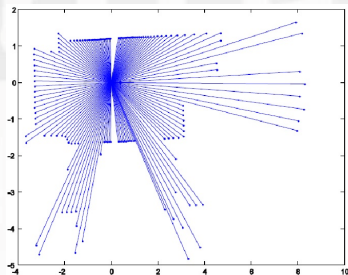
$$A\bar{x} = B \quad (12)$$

where:

$$A = \begin{pmatrix} 2(x_n - x_1) & 2(y_n - y_1) \\ 2(x_n - x_2) & 2(y_n - y_2) \\ \vdots & \vdots \\ 2(x_n - x_{n-1}) & 2(y_n - y_{n-1}) \end{pmatrix}, \quad \bar{x} = [x \ y]^T$$

Motivation

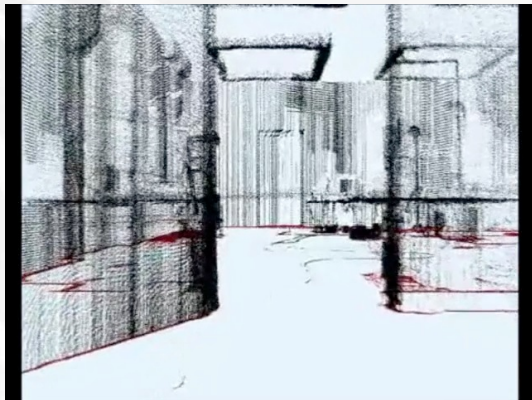
- Point Cloud $\Rightarrow$ extract Lines / Planes
- Why Features:
 - Raw data: huge amount of data to be stored
 - Compact features require less storage
 - Provide rich and accurate information
 - Basis for high level features (e.g. more abstract features, objects)
- Here, we will study line segments
 - The simplest geometric structure
 - Suitable for most office-like environments



Uncertainty representation | line extraction

Extract lines from a point cloud (e.g. range scan)

- Three main problems:
 - How many lines are there?
 - **Segmentation**: Which points belong to which line?
 - **Line Fitting/Extraction**: Given points that belong to a line, how to estimate the line parameters?
- Algorithms we will see:
 - ① Split-and-merge
 - ② Linear regression
 - ③ RANSAC
 - ④ Hough-Transform



Line extraction | 1. split-and-merge (standard)

- Popular algorithm, originates from Computer Vision.
- A recursive procedure of fitting and splitting.
- A slightly different version, called Iterative end-point-fit, simply connects the end points for line fitting.

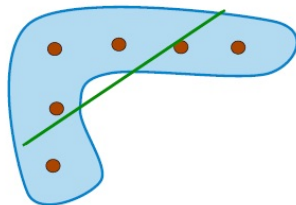
Let S be the set of all data points

Split

- Fit a line to points in current set S
- Find the most distant point to the line
- If distance $>$ threshold $\Rightarrow$ split set & repeat with left and right point sets

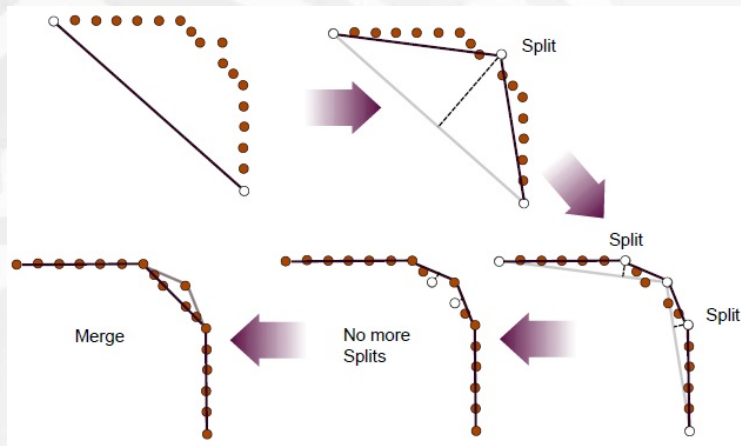
Merge

- If two consecutive segments are collinear enough, obtain the common line and find the most distant point
- If distance $\leq$ threshold, merge both segments



Line extraction | 1. split-and-merge (iterative end-point-fit)

- Iterative end-point-fit:
simply connects the end points for line fitting

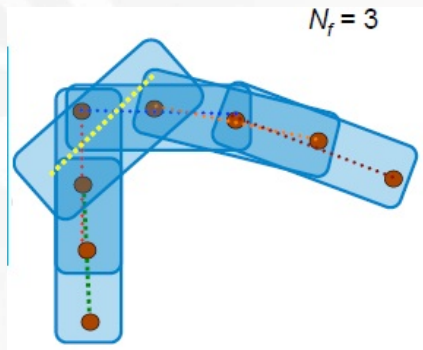


Line extraction | 2. line-regression

- “Sliding window” of size N_f points
- Fit line-segment to all points in each window

Line-Regression

- Initialize sliding window size N_f
- Fit a line to every N_f consecutive points (i.e. in each window)
- Merge overlapping line segments + re-compute line parameters for each segment



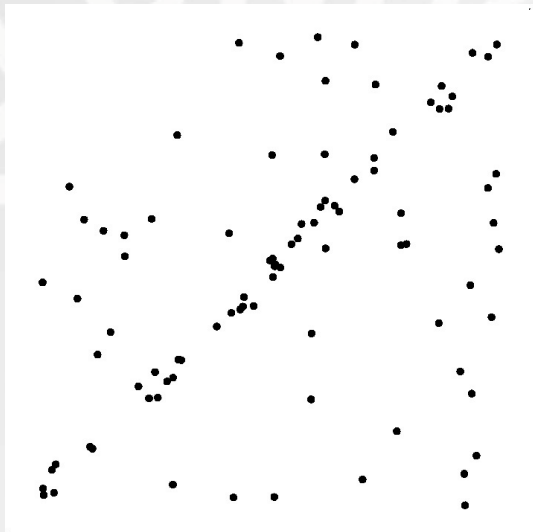
Line extraction | 3. RANSAC

- **RANSAC = RANdom SAmple Consensus.**

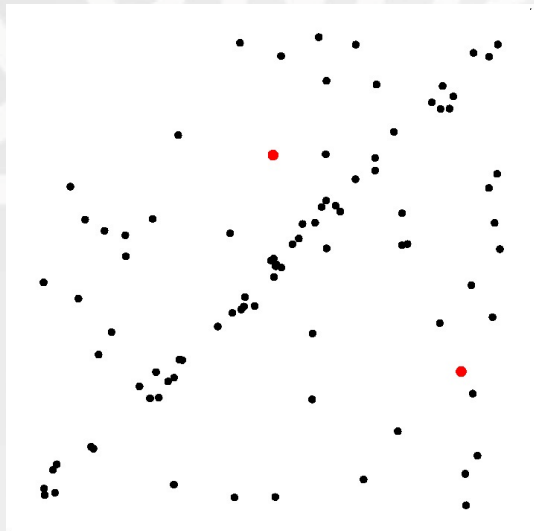
M. Fischler, and R. C. Bolles. Random sample consensus: *A paradigm for model fitting with applications to image analysis and automated cartography*. Graphics and Image Processing, 1981.

- A generic & robust fitting algorithm of models in the presence of outliers (i.e. points which do not satisfy a model)
- Can be applied in general to any problem where the goal is to **identify the inliers which satisfy a predefined model**.
- Typical applications in robotics are:
line extraction from 2D range data, plane extraction from 3D data, feature matching, structure from motion, camera calibration, homography estimation, etc.
- RANSAC is **iterative and non-deterministic** $\Rightarrow$ the probability to find a set free of outliers increases as more iterations are used
- Drawback: a non-deterministic method, results are different between runs.

Line extraction | 3. RANSAC

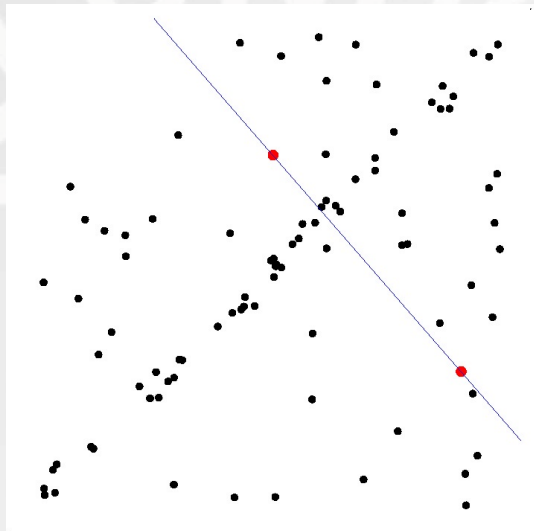


Line extraction | 3. RANSAC



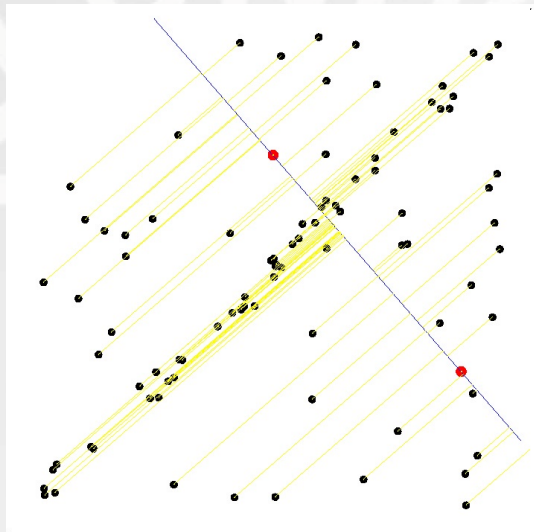
- Select sample of 2 points at random

Line extraction | 3. RANSAC



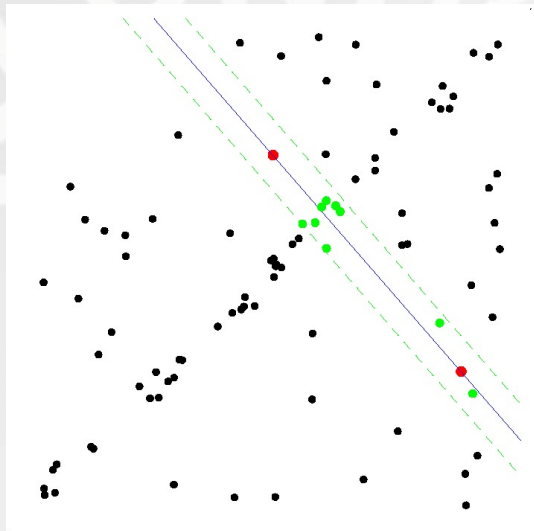
- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample

Line extraction | 3. RANSAC



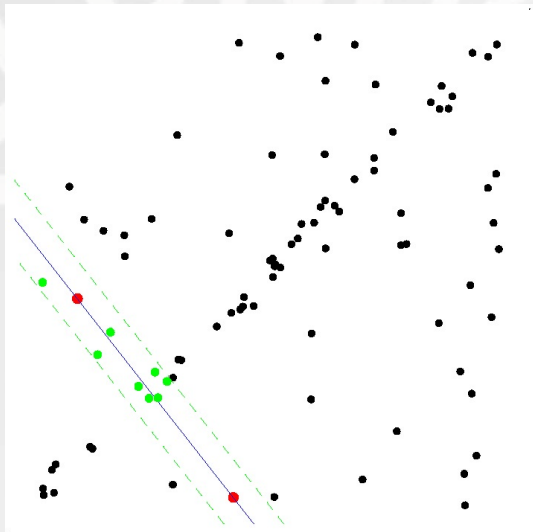
- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point

Line extraction | 3. RANSAC



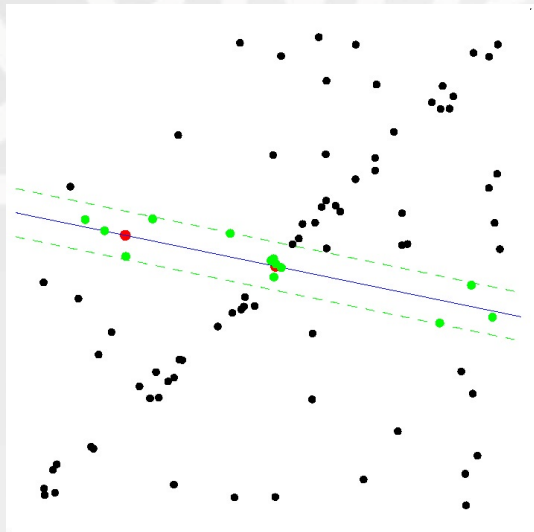
- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis

Line extraction | 3. RANSAC



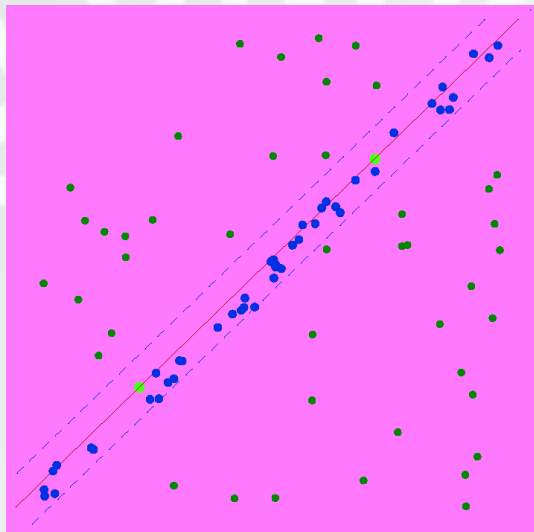
- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis
- Repeat sampling

Line extraction | 3. RANSAC



- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis
- Repeat sampling

Line extraction | 3. RANSAC



- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis
- Repeat sampling
- Set with the maximum number of inliers obtained after k iterations

Line extraction | 3. RANSAC

How many iterations does RANSAC need?

- Ideally: check all possible combinations of 2 points in a dataset of N points.
- Number of all pairwise combinations: $N(N - 1)/2$
 $\Rightarrow$ a computationally unfeasible if N is too large.
example: 10,000 points to fit a line through a need to check all $10,000 * 9999/2 = 50$ million combinations!
- Do we really need to check all combinations or can we stop after some iterations?
Checking a **subset** of combinations is enough if we have a **rough** estimate of the percentage of inliers in our dataset
- This can be done in a probabilistic way

Line extraction | 3. RANSAC

How many iterations does RANSAC need?

- $w := \text{number of inliers} / N$ and $N := \text{tot. no. data points}$
 $\Rightarrow w$: fraction of inliers in the dataset
 $\Rightarrow w = P(\text{selecting an inlier-point out of the dataset})$
- Let $p := P(\text{selecting a minimal set of points free of outliers})$
- **Assumption:** the 2 points necessary to estimate a line are selected independently
 $\Rightarrow w^2 = P(\text{both selected points are inliers})$
 $\Rightarrow 1 - w^2 = P(\text{at least one of these two points is an outlier})$
- Let $k := \text{no. RANSAC iterations executed so far}$
 $\Rightarrow (1 - w^2)^k = P(\text{RANSAC never selects two points that are both inliers})$
 $\Rightarrow 1 - p = (1 - w^2)^k$, and therefore:

$$k = \frac{\log(1 - p)}{\log(1 - w^2)}$$

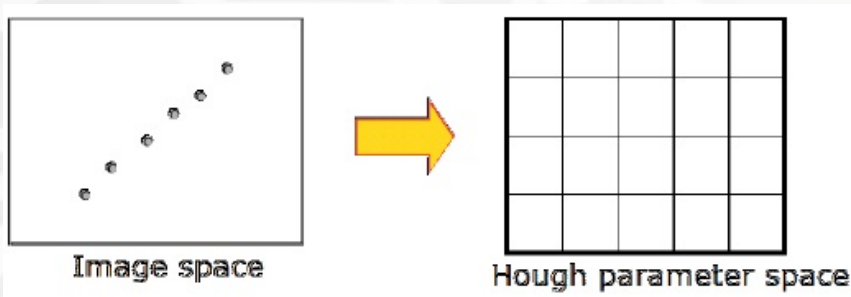
Line extraction | 3. RANSAC

How many iterations does RANSAC need?

- The number of iterations k is: $k = \frac{\log(1 - p)}{\log(1 - w^2)}$
 $\Rightarrow$ knowing the fraction of inliers w , after k RANSAC iterations we will have a probability p of finding a set of points free of outliers
- Example: if we want a probability of success $p = 99\%$ and we know that $w = 50\% \Rightarrow k = 16$ iterations -these are dramatically fewer than the number of all possible combinations!
- Notice: the number of points does not influence the estimated number of iterations, only w does!
- In practice we need only a rough estimate of w .
More advanced variants of RANSAC estimate the fraction of inliers and adaptively change it on every iteration

Line extraction | 4. Hough-Transform

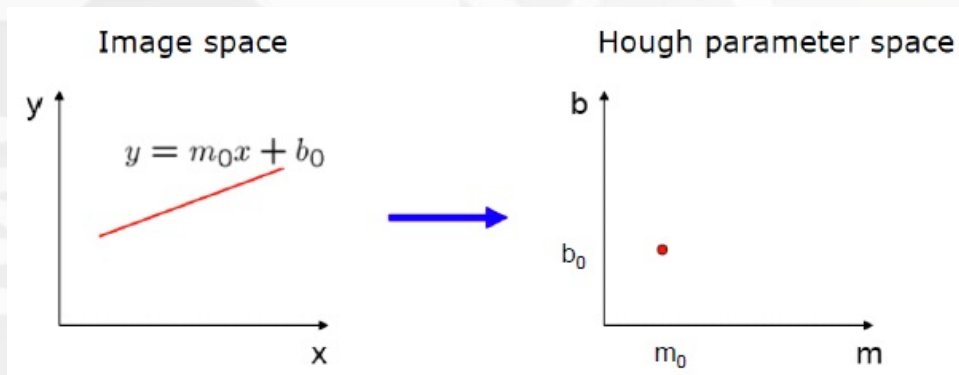
- Points vote for plausible line parameters
- Hough-Transform: maps image-space into Hough-space
- **Hough-space**: voting accumulator, parametrized w.r.t. line characteristics



- 1 P. Hough, *Machine Analysis of Bubble Chamber Pictures*, Proc. Int. Conf. High Energy Accelerators and Instrumentation, 1959
- 2 J. Richard, O. Duda, P.E. Hart (April 1971). *Use of the Hough Transformation to Detect Lines and Curves in Pictures*. Artificial Intelligence Center (SRI International)

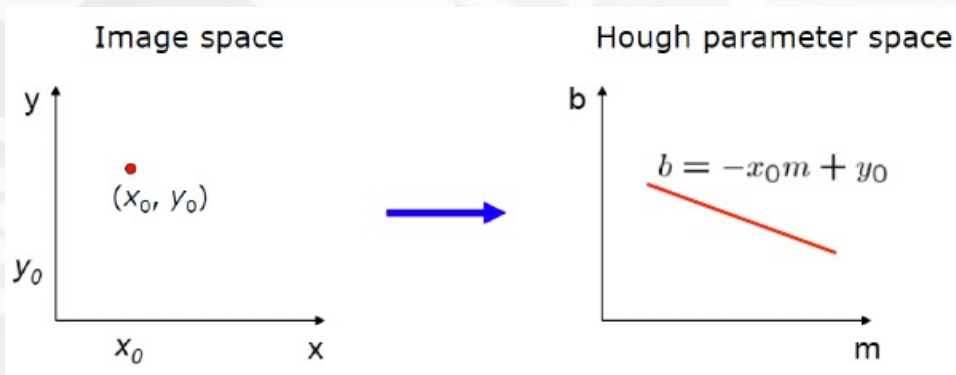
Line extraction | 4. Hough-Transform

- A line in the image corresponds to a point in Hough space



Line extraction | 4. Hough-Transform

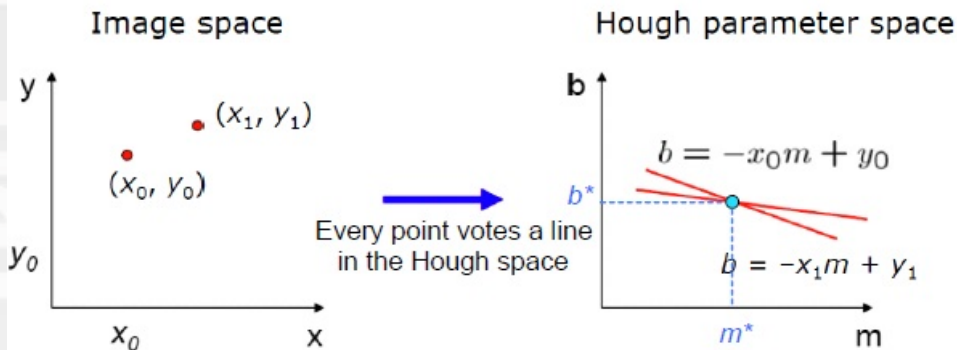
- A line in the image corresponds to a point in Hough space
- What does a point (x_0, y_0) in the image space map to in the Hough space?



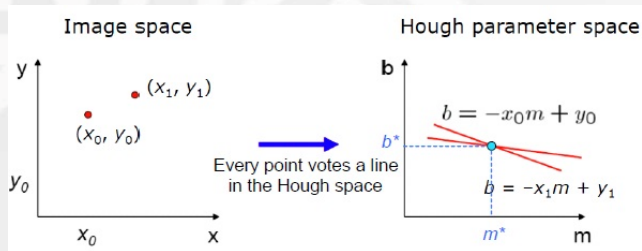
Line extraction | 4. Hough-Transform

- A line in the image corresponds to a point in Hough space
- What does a point (x_0, y_0) in the image space map to in the Hough space?
- Where is the line that contains both (x_0, y_0) and (x_1, y_1) ?

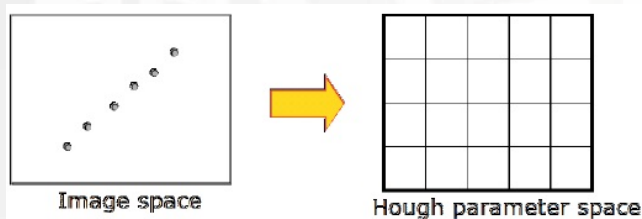
It is the intersection of the lines $b = -x_0m + y_0$ and $b = -x_1m + y_1$



Line extraction | 4. Hough-Transform



- Each point in image space, votes for line-parameters in Hough parameter space



Line extraction | 4. Hough-Transform

- Problems with the (m, b) space:
 - Unbounded parameter domain
 - How to represent lines aligned with the axes?
- Alternative: **polar representation**

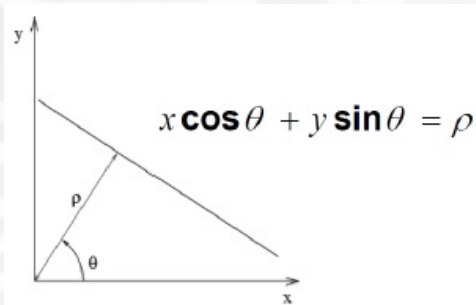


图 3.1: Each point in image space will map to a sinusoid in the (ρ, θ) parameter space

Line extraction | 4. Hough-Transform

Step 1: Initialize accumulator H to all zeros

Step 2: **for** each edge point (x, y) in the image

for all θ in $[0, 180]$

- Compute $\rho = x \cos \theta + y \sin \theta$
- $H(\theta, \rho) = H(\theta, \rho) + 1$

end

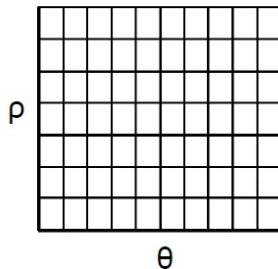
end

Step 3: Find the values of (θ, ρ) where $H(\theta, \rho)$ is a local maximum

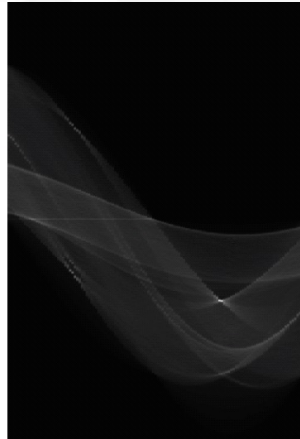
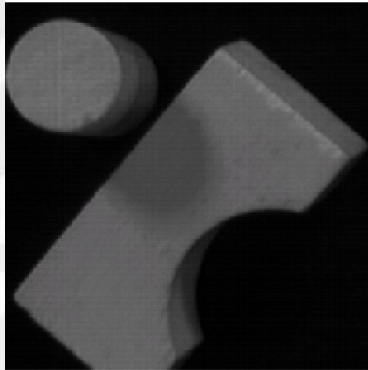
Step 4: The detected line in the image is given by:

$$\rho = x \cos \theta + y \sin \theta$$

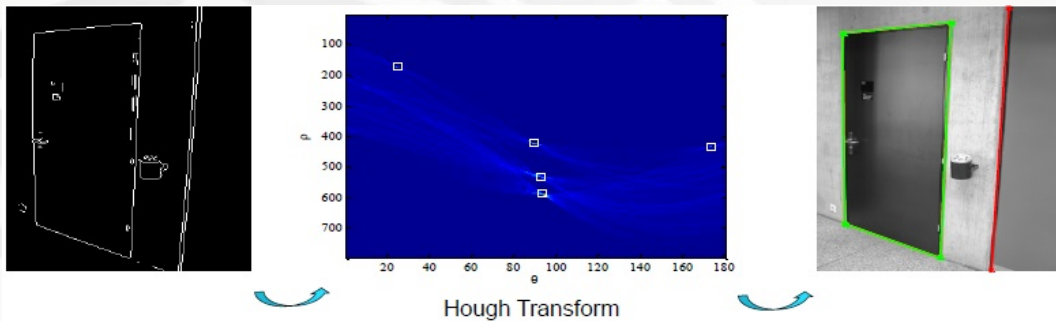
H: accumulator array (votes)



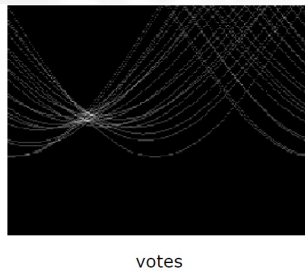
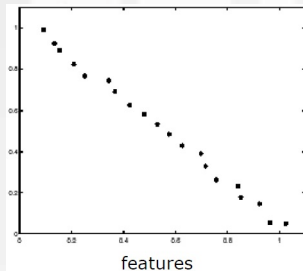
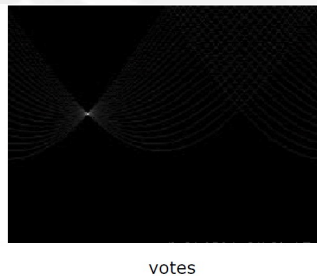
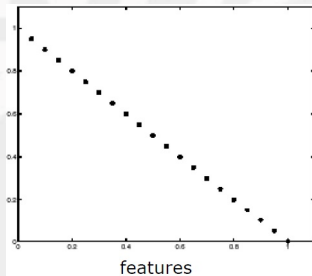
Line extraction | 4. Hough-Transform: examples



Line extraction | 4. Hough-Transform: examples



Line extraction | 4. Hough-Transform: examples



Line extraction | comparison

- Split-and-merge, Incremental and Line-Regression: fastest - best applied on laser scans
- Deterministic & make use of the sequential ordering of raw scan points (points captured according to the rotation direction of the laser beam)
- If applied on randomly captured points only last 2 algorithms would segment all lines.
- RANSAC and Hough-Transform produce greater precision $\Rightarrow$ a more robust to outliers

	Complexity	Speed (Hz)	False positives	Precision
Split-and-Merge	$N \log N$	1500	10%	+++
Incremental	SN	600	6%	+++
Line-Regression	NN_f	400	10%	+++
RANSAC	SNk	30	30%	++++
Hough-Transform	$SN N_C + SN_R N_C$	10	30%	++++

最小二乘直线拟合

- Point-Line distance

$$\rho_i \cos(\theta_i - \alpha) - r = d_i$$

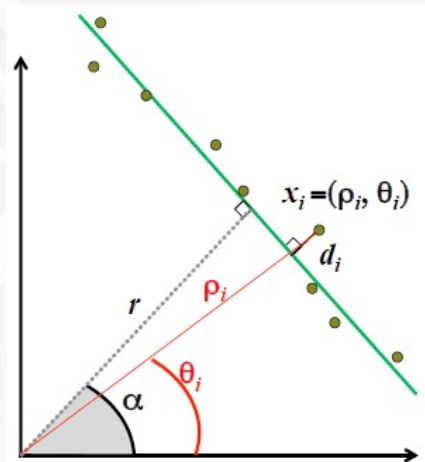
- If each measurement is equally uncertain then sum of sq. errors:

$$S = \sum_i d_i^2 = \sum_i (\rho_i \cos(\theta_i - \alpha) - r)^2$$

- Goal: minimize S when selecting $(r, \alpha) \Rightarrow$ solve the system

$$\frac{\partial S}{\partial \alpha} = 0 \quad \frac{\partial S}{\partial r} = 0$$

- “Unweighted Least Squares”



激光雷达传感器特性

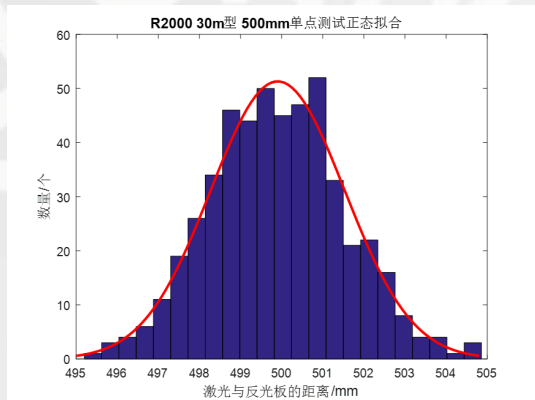


图 3.2: 定距下激光雷达测量误差统计

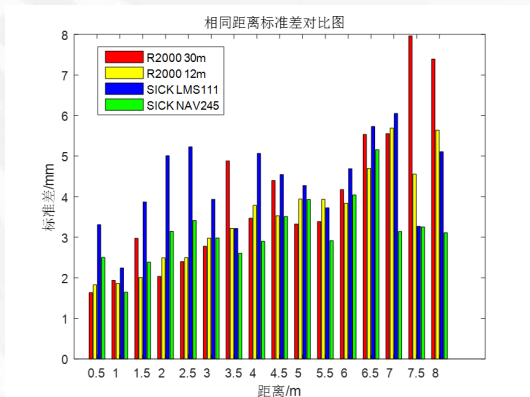


图 3.3: 激光雷达测量误差随距离的变化

加权最小二乘直线拟合

- Point-Line distance

$$\rho_i \cos(\theta_i - \alpha) - r = d_i$$

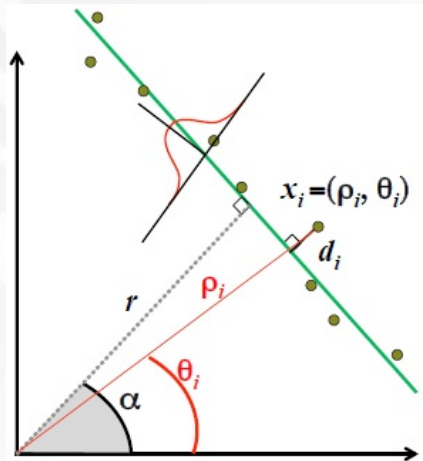
- Each sensor measurement, may have its own, unique uncertainty σ^2

$$S = \sum_i w_i d_i^2 = \sum_i w_i (\rho_i \cos(\theta_i - \alpha) - r)^2$$

$$w_i = \frac{1}{\sigma^2}$$

- “Weighted Least Squares”

$$\frac{\partial S}{\partial \alpha} = 0 \quad \frac{\partial S}{\partial r} = 0$$



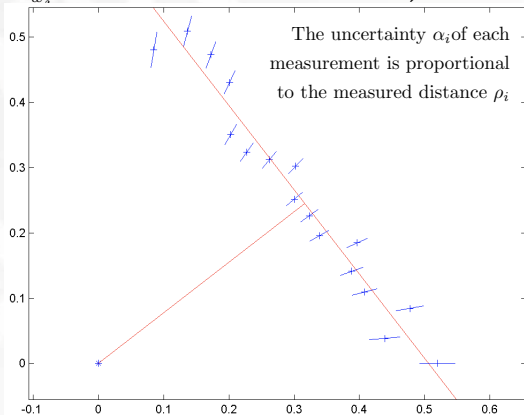
加权最小二乘直线拟合

- Gives the line parameters:

$$\alpha = \frac{1}{2} \arctan \left(\frac{\sum w_i \rho_i^2 \sin 2\theta_i - \frac{2}{\sum w_i} \sum \sum w_i w_j \rho_i \rho_j \cos \theta_i \sin \theta_j}{\sum w_i \rho_i^2 \cos 2\theta_i - \frac{1}{\sum w_i} \sum \sum w_i w_j \rho_i \rho_j \cos(\theta_i + \theta_j)} \right)$$

$$r = \frac{\sum w_i \rho_i \cos(\theta_i - \alpha_i)}{\sum w_i}$$

- If $\begin{cases} \rho_i \sim N(\hat{\rho}_i, \sigma_{\rho_i}^2) \\ \theta_i \sim N(\hat{\theta}_i, \sigma_{\theta_i}^2) \end{cases}$ what is the uncertainty in the line (r, α) ?



加权最小二乘直线拟合的误差传导模型

- The uncertainty of **each measurement** $x_i = (\rho_i, \theta_i)$ is described by the covariance matrix:

$$C_{x_i} = \begin{bmatrix} \sigma_{\rho_i}^2 & 0 \\ 0 & \sigma_{\theta_i}^2 \end{bmatrix} \quad \text{Assuming that } \rho_i, \theta_i \text{ are independent}$$

- The uncertainty in the line (r, α) is described by the covariance matrix:

$$C_{r\alpha} = \begin{bmatrix} \sigma_{\alpha}^2 & \sigma_{\alpha r} \\ \sigma_{r\alpha} & \sigma_r^2 \end{bmatrix}$$

Define:

$$C_x = \begin{bmatrix} \text{diag}(\sigma_{\rho}^2) & 0 \\ 0 & \text{diag}(\sigma_{\theta}^2) \end{bmatrix} = \begin{bmatrix} \sigma_{\rho_1}^2 & 0 & \cdots & 0 & 0 \\ 0 & \sigma_{\rho_2}^2 & \cdots & 0 & 0 \\ 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & \cdots & \sigma_{\theta_{n-1}}^2 & 0 \\ 0 & 0 & \cdots & 0 & \sigma_{\theta_n}^2 \end{bmatrix}_{2n \times 2n}$$

加权最小二乘直线拟合的误差传导模型

$$C_x = \begin{bmatrix} \text{diag}(\sigma_\rho^2) & 0 \\ 0 & \text{diag}(\sigma_\theta^2) \end{bmatrix} = \begin{bmatrix} \sigma_{\rho_1}^2 & 0 & \dots & 0 & 0 \\ 0 & \sigma_{\rho_2}^2 & \dots & 0 & 0 \\ 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & \dots & \sigma_{\theta_{n-1}}^2 & 0 \\ 0 & 0 & \dots & 0 & \sigma_{\theta_n}^2 \end{bmatrix}_{2n \times 2n}$$

Jacobian:

$$F_{\rho\theta} = \begin{bmatrix} \frac{\partial \alpha}{\partial \rho_1} & \frac{\partial \alpha}{\partial \rho_2} & \dots & \frac{\partial \alpha}{\partial \theta_{n-1}} & \frac{\partial \alpha}{\partial \theta_n} \\ \frac{\partial r}{\partial \rho_1} & \frac{\partial r}{\partial \rho_2} & \dots & \frac{\partial r}{\partial \theta_{n-1}} & \frac{\partial r}{\partial \theta_n} \end{bmatrix}$$

From Error Paopagation Law:

$$C_{\alpha r} = F_{\rho\theta} C_x F_{\rho\theta}^T$$



作业

直线特征提取仿真实验

- 设计并实现一种基于 2D 点云的直线提取算法
- 通过仿真实验，并对仿真实验结果进行分析，测试所构建模型的正确性
- 将模型的推导过程，仿真实验结果和数据分析补充到打报告之后，提交电子版文档，文件名称：学号 + 专业 + 姓名
- 4 月 26 日晚上 23:59 之前发送邮箱：1537664323@qq.com

第五章 移动机器人感知技术

激光传感器定位原理

孙 雷 教授

sunl@nankai.edu.cn

Tel: 13512967601

机器人与信息自动化研究所 (IRAIS)
天津市智能机器人技术重点实验室 (tjKLIR)
南开大学

2026 年 4 月 13 日